ML Team Week 1 - HW1

Tuesday, March 10, 2026

Homework Practice Sheet (Intro ML / Neural Nets)
Forward Pass, Backpropagation, Gradient Descent + Concepts

Instructions

Show your work. Unless otherwise stated, use:

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \mathrm{ReLU}(z) = \max(0, z),$$

$$L_{\mathrm{MSE}} = \frac{1}{2}(\hat{y} - y)^2, \qquad \text{and} \qquad w \leftarrow w - \eta \frac{\partial L}{\partial w}.$$

For conceptual questions, answer in 1–4 sentences.

Part A: Numeric Problems (Forward Pass / Backprop / Updates)

1. **Forward pass (2 inputs, linear neuron).**
   Given $x = \begin{bmatrix} 2 \\ -1 \end{bmatrix}$, $w = \begin{bmatrix} 1 \\ 3 \end{bmatrix}$, $b = -2$. Compute:

$$z = w^\top x + b.$$

1. $z = \begin{bmatrix} 1 & 3 \end{bmatrix} \begin{bmatrix} 2 \\ -1 \end{bmatrix} + (-2)$

$z = (2-3) + (-2) = \boxed{-3}$

2. **Forward pass (sigmoid neuron).**
   Given $x = 4$, $w = -0.5$, $b = 1$. Compute:

$$z = wx + b, \qquad \hat{y} = \sigma(z).$$

2. $z = 4(-0.5) + 1 = -2 + 1 = -1$

$\hat{y} = \sigma(-1) = \frac{1}{1+e^{1}} = \frac{1}{1+e} \approx \boxed{0.2689}$

3. **Forward pass (tiny 1-hidden-layer net).**
   Let

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad W_1 = \begin{bmatrix} 1 & -1 \\ 2 & 0 \end{bmatrix}, \quad b_1 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

   Compute:

$$z_1 = W_1 x + b_1, \qquad h = \mathrm{ReLU}(z_1).$$

   Then with

$$W_2 = \begin{bmatrix} 2 & -1 \end{bmatrix}, \quad b_2 = 0,$$

   compute:

$$y = W_2 h + b_2.$$

3. $z_1 = \begin{bmatrix} 1 & -1 \\ 2 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 2 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 3 \end{bmatrix}$

$h = \mathrm{ReLu}\left(\begin{bmatrix} -1 \\ 3 \end{bmatrix}\right) = \begin{bmatrix} 0 \\ 3 \end{bmatrix}$

$y = \begin{bmatrix} 2 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 3 \end{bmatrix} + 0 = -3 + 0 = \boxed{-3}$

4. $L = \frac{1}{2}(-1-2)^2 = \frac{1}{2} \cdot 9 = \boxed{\frac{9}{2}}$

4. **MSE loss.**
Given prediction $\hat{y} = -1$ and target $y = 2$, compute:

$$L = \frac{1}{2}(\hat{y} - y)^2.$$

5. $\frac{dL}{d\hat{y}} = (\hat{y} - y) \cdot 1 = \boxed{\hat{y} - y}$

5. **Backprop warm-up (MSE derivative).**
For $L = \frac{1}{2}(\hat{y} - y)^2$, compute $\frac{\partial L}{\partial \hat{y}}$ in terms of $\hat{y}$ and y.

6. $L = \frac{1}{2}(w^T x + b - y)^2$
$= \frac{1}{2}(w_1 x_1 + w_2 x_2 + b - y)^2$

6. **Backprop for a linear neuron (2 inputs).**
Model: $\hat{y} = w^\top x + b$, loss $L = \frac{1}{2}(\hat{y} - y)^2$.
Given:

$$x = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, \quad w = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad b = 1, \quad y = 0.$$

Compute $\frac{\partial L}{\partial w_1}$, $\frac{\partial L}{\partial w_2}$, and $\frac{\partial L}{\partial b}$.

$dL/db = (w_1 x_1 + w_2 x_2 + b - y) = \boxed{-1}$

$dL/dw_1 = (w_1 x_1 + w_2 x_2 + b - y) * x_1 = 0 - 2 + 1 - 0 = -1$

$dL/dw_2 = (w_1 x_1 + w_2 x_2 + b - y) * x_2 = (-1) * (-2) = 2$

7. **One gradient descent update (linear neuron).**
Use the same setup as Problem 6 with learning rate $\eta = 0.2$. Compute the updated w and b.

7. $w_1^{new} = w_1 - 0.2\left(\frac{dL}{dw_1}\right) = 0 + 0.2 = 0.2$
$w_2^{new} = w_2 - 0.2\left(dL/dw_2\right) = 1 - 0.4 = 0.6$
$w^{new} = \begin{bmatrix} 0.2 \\ 0.6 \end{bmatrix}$
$b^{new} = 1 - 0.2(-1) = 1 + 0.2 = 1.2$

8. **ReLU backprop (single neuron).**
Let $z = wx + b$, $a = \text{ReLU}(z)$, and $L = \frac{1}{2}(a - y)^2$.
Given $x = -1$, $w = 2$, $b = -1$, $y = 3$.
Compute z, a, and then $\frac{\partial L}{\partial w}$ and $\frac{\partial L}{\partial b}$. (Be careful about the ReLU derivative.)

8.
$z = (-1)(2) + (-1) = -3$
$a = \text{ReLU}(-3) = 0$
$\frac{dL}{dw} = (\text{ReLU}(wx+b) - y) \cdot \text{ReLU}'(wx+b) \cdot x = (-3)(1)(-1) = 3$
$\frac{dL}{db} = (\text{ReLU}(wx+b) - y)\,(\text{ReLU}'(wx+b)) = (-3)(1) = -3$

9. **Two-step gradient descent (linear regression).**
Data point $(x, y) = (2, 1)$, initialize $w_0 = 0$, $b_0 = 0$, learning rate $\eta = 0.1$.
Run **two** steps of gradient descent using:

$$\hat{y} = wx + b, \quad L = \frac{1}{2}(\hat{y} - y)^2.$$

Report (w_1, b_1) and (w_2, b_2).

9.
$w_1 = w_0 - \eta(w_0 x + b_0 - y)x = 0 - 0.1(-1)(2) = 0.2$
$b_1 = b_0 - \eta(w_0 x + b_0 - y) = 0 - 0.1(0 + 0 - 1) = 0.1$
$(0.2, 0.1)$
$w_2 = 0.2 - 0.1(0.4 + 0.1 - 1)(2) = 0.3$
$b_2 = 0.1 - 0.1(0.4 + 0.1 - 1) = 0.15$
$(0.3, 0.15)$

10. **Softmax probabilities (3 classes).**
Given logits $s = \begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix}$. Compute softmax$(s)$. (You may leave answers in terms of e.)

$$\text{softmax}\left(\begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix}\right) = \left[\frac{1}{1+e+e^2}, \frac{e}{1+e+e^2}, \frac{e^2}{1+e+e^2}\right]^T$$

Part B: Conceptual Questions (Short Answers)

11. **Forward pass vs. backprop.**
In your own words, what happens during the forward pass, and what happens during backpropagation?

12. **Why do we need activation functions?**
What problem would occur if a neural network used *no* nonlinear activation functions (only linear layers)?

13. **Learning rate intuition.**
What can happen if the learning rate is (a) too large, or (b) too small?

14. **Vanishing gradients (high level).**
What does "vanishing gradients" mean, and why can it make training deep networks difficult?

15. **ReLU vs. sigmoid (practical).**
Give one reason ReLU is often preferred over sigmoid for hidden layers.

11.

During the forward pass, input(s) from the previous layer and weights are used for a dot product and then we add bias (weights and biases are initially randomly generate). Then, we do the activation function with the intermediate value.

During backprop, we update each bias and weight in the network. We do this by updating the current weight by subtracting a learning rate * (delta * transposed activations from previous layer).

(previous activation - target) * f'(x)

The current bias is updated by subtracting a learning rate * delta.

12. No nonlinear functions means we can only have linear layers. No matter how layers you stack, it'll still be linear mappings, which means you can't capture nonlinear relationships between neurons.

13. If learning rate is too large, we can move past the local minimum when updating weights and biases. If learning rate is too small, the weights and biases will barely change from layer to layer.

14. "Vanishing gradients" mean that the updates to the weights becomes really small, therefore, the weights barely change during training. That means the training accuracy and test accuracy would stay the same between epochs.

15. Negative neurons are flattered to zero, avoiding the vanishing gradient problem.

sigmoid can still have negative neurons.